

Project 1 - Socket Programming

1. วัตถุประสงค์ของโปรแกรม

โปรแกรมที่พัฒนาคือ Webtoon Reader ซึ่งเป็น Network Application แบบ Client-Server สำหรับอ่านการ์ตูนออนไลน์ (Webtoon) โดย Server เก็บข้อมูลชุดการ์ตูน (series) ตอน (episode) และหน้าเนื้อหา (page) ไว้ ส่วน Client ใช้เชื่อมต่อเพื่อ:

- ดูรายชื่อและค้นหาการ์ตูนที่มีในระบบ
- ดูรายการตอนของการ์ตูนที่เลือก
- อ่านเนื้อหาแต่ละหน้าของตอน แบบโหลดทุกหน้าตามลำดับอัตโนมัติ

2. ความแตกต่างจากโปรแกรมที่มีอยู่ในปัจจุบัน

เว็บอ่าน Webtoon ทั่วไปบางเว็บทำงานผ่าน HTTP ที่ไม่ใช่ HTTP/3 ซึ่งอาจใช้ UDP ทำให้การส่งข้อมูลรูปภาพมาไม่ตรงตามลำดับ เพราะ UDP เป็น protocol ที่ไม่ได้การันตีว่า packet จะส่งมาตามลำดับ โปรแกรมที่นำเสนอนี้แตกต่างออกไปตรงที่เราจะใช้ TCP ในการส่งแทน เพื่อให้มี reliability และให้ packet ที่ได้มาเป็น in-order เพื่อให้ส่งหน้าแรกไปก่อนเรื่อย ๆ ตามลำดับ

3. คุณลักษณะของแอปพลิเคชัน

คุณลักษณะ	รายละเอียด
Network Application Architecture	Client-Server: Server เก็บข้อมูลกลาง Client หลายเครื่องเชื่อมต่อพร้อมกันได้
Data Transfer	ข้อความแบบ text-based protocol (คล้าย HTTP) ส่ง metadata และเนื้อหาหน้า
Reliability Requirement	ต้องการความถูกต้องครบถ้วนของข้อมูล 100% ห้ามหน้าขาดหรือสลับลำดับ
Timing Requirement	ไม่ real-time เข้มงวดแบบ VoIP/เกม หน่วงเวลาได้เล็กน้อยระหว่างโหลดหน้า
Session	ไม่รักษาสถานะระหว่าง client-server
Concurrency	Server รองรับหลาย client พร้อมกัน (multi-threaded)

4. การเลือก Transport Layer Service Model: TCP

โปรแกรมนี้เลือกใช้ TCP แทน UDP โดยมีเหตุผลดังนี้:

- Reliable data transfer: เนื้อหาการ 전송ต้องได้รับครบถ้วนไม่สูญหายหรือผิดพลาด หากหน้าภาพขาดหรือ byte สลับลำดับ ผู้อ่านจะเห็นข้อมูลผิดพลาดทันที UDP ไม่มีการรับประกันการส่งถึงจึงไม่เหมาะสม
- In-order delivery: โปรโตคอลออกแบบเป็น stream ข้อความต่อเนื่องแบบ request-response จำเป็นต้องได้รับข้อมูลตามลำดับที่ส่ง ซึ่ง TCP จัดการให้อัตโนมัติ
- Connection-oriented: เหมาะกับ session ที่ client เปิดการเชื่อมต่อค้างไว้แล้วส่งหลาย request ต่อเนื่องกัน เช่น ดูรายการ เลือกตอน แล้วอ่านหน้าต่อหน้า
- ไม่มีข้อจำกัดด้าน delay/throughput แบบ real-time ที่ต้องยอมเสียข้อมูลบางส่วนเพื่อความเร็วแบบวิดีโอคอลหรือเกมออนไลน์ จึงไม่จำเป็นต้องใช้ UDP
- TCP มีกลไก flow control และ congestion control ในตัว ช่วยจัดการเมื่อเครือข่ายหนาแน่นโดยไม่ต้องเขียนกลไกเองใน application layer

5. การออกแบบ Application-Layer Protocol: WTRP (WebToon Retrieval Protocol) v1.0

WTRP เป็นโปรโตคอลแบบ text-based คล้าย HTTP ที่ออกแบบขึ้นเองใหม่ทั้งหมด รันอยู่บน TCP โดยตรง (ไม่ผ่าน HTTP)

5.1 รูปแบบ Request

```
<METHOD> <RESOURCE-PATH> WTRP/1.0
```

```
Header1: value
```

```
Header2: value
```

```
[BODY (ถ้ามี)]
```

5.2 รูปแบบ Response

WTRP/1.0 <STATUS-CODE> <STATUS-PHRASE>

Header1: value

Header2: value

[BODY (ถ้ามี)]

5.3 Methods ที่รองรับ

Method	Path ตัวอย่าง	ความหมาย
LIST_SERIES	/series	ขอรายชื่อการ์ตูนทั้งหมด
GET_EPISODE	/series/{id}/ep/{no}	ขอ metadata ของตอน (จำนวนหน้า ฯลฯ)
GET_PAGE	/series/{id}/ep/{no}/page/{p}	ขอเนื้อหาหน้าที่ p

5.4 Status Code

Code	Phrase	ความหมาย
200	OK	สำเร็จ
206	Partial Content	ส่งข้อมูลหน้าเดียวจากหลายหน้า
400	Bad Request	รูปแบบ request ผิด
404	Not Found	ไม่พบ series/episode/page
405	Method Not Allowed	Method ไม่รู้จัก
500	Internal Server Error	ข้อผิดพลาดภายใน server

5.5 ตัวอย่างการสนทนาจริง

Client -> Server:

GET_PAGE /series/1/ep/1/page/2 WTRP/1.0

Client-ID: c001

Server -> Client:

WTRP/1.0 206 Partial Content

Series-Id: 1

Episode: 1

Page: 2

Total-Pages: 5

Content-Length: 42

[หน้า 2 จากตอนที่ 1 ของเรื่อง Moon Hunter]

6. การพัฒนาโปรแกรม (Implementation)

โปรแกรมพัฒนาด้วยภาษา Python โดยใช้ socket module มาตรฐาน ประกอบด้วย 2 ไฟล์หลัก:

- server.py — WTRP Server แบบ multi-threaded รับบน TCP รองรับหลาย client พร้อมกัน เก็บข้อมูล series/episode/page
- client.py — WTRP Client แบบโต้ตอบผ่านเมนู (interactive CLI) ส่ง request และแสดงผล response พร้อม status code/phrase ทุกครั้ง

คุณสมบัติเพิ่มเติมที่พัฒนา:

- เมนูที่ 2: อ่านทั้งตอนอัตโนมัติ (Read full episode) — client จะเรียก GET_EPISODE เพื่อทราบจำนวนหน้าทั้งหมดก่อน แล้ววนส่ง GET_PAGE ที่ละหน้าตามลำดับเริ่มจากหน้าที่ 1 จนถึงหน้าสุดท้าย บน connection TCP เดียวกัน

6.1 ตัวอย่างผลลัพธ์การรันโปรแกรม (Client)

```
Connected to WTRP server at 127.0.0.1:5050 (TCP)

WebToon Reader (WTRP Client)
-----
1) List all series
2) Read an episode
9) Quit
Choose an option: 1

=== Sending Request ===
LIST_SERIES /series WTRP/1.0
=== Received Response ===
Status: 200 OK
Content-Length: 96
Body:
1|Moon Hunter|J. Kim|Fantasy|2 episodes
2|Cafe of Small Miracles|S. Park|Slice of Life|1 episode
=====
```

รูปที่ 6.11 ผลลัพธ์การรันโปรแกรม client

6.2 ตัวอย่างผลลัพธ์การรันโปรแกรม (Server)

```
WTRP Server listening on 0.0.0.0:5050 (TCP)
[+] Client connected: 127.0.0.1:7016

--- Received from 127.0.0.1:7016 ---
LIST_SERIES /series WTRP/1.0
--- Sent to 127.0.0.1:7016 ---
WTRP/1.0 200 OK
```

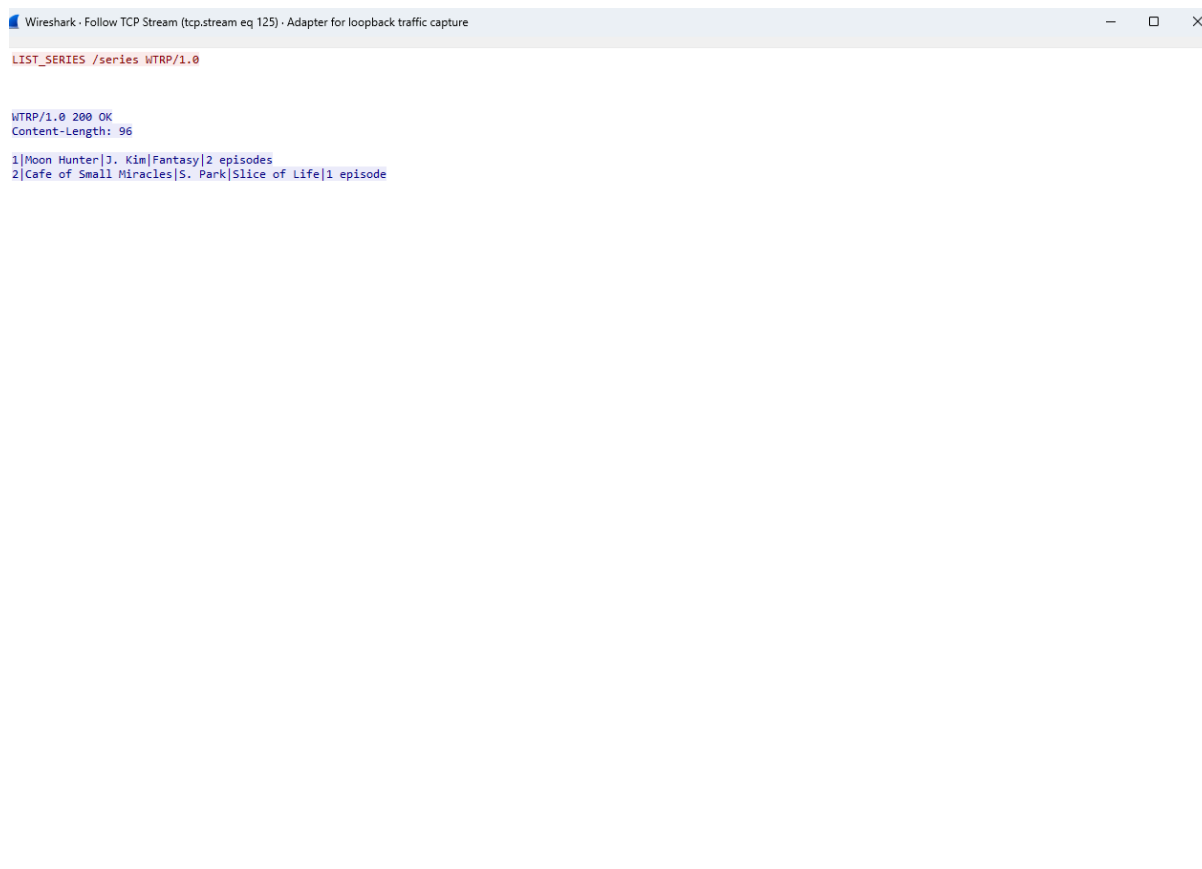
รูปที่ 6.21 ผลลัพธ์การรันโปรแกรม server

7. การตรวจสอบด้วย Wireshark

เนื่องจาก WTRP สื่อสารด้วย plain text ผ่าน TCP socket โดยตรง ไม่มีการเข้ารหัส (ไม่ใช่ HTTPS/TLS) จึงสามารถใช้ Wireshark ดักจับ packet และอ่านเนื้อหา request/response ได้โดยตรง โดยตั้ง capture/display filter เป็น tcp.port == 5050 แล้วใช้ฟังก์ชัน Follow -> TCP Stream เพื่อดูข้อความเต็มรูปแบบของการสนทนาแต่ละ connection ซึ่งจะเห็น TCP 3-way handshake (SYN, SYN-ACK, ACK) และ payload ที่เป็นข้อความ WTRP ทั้ง method, status code/phrase, header และ body ได้อย่างชัดเจน

No.	Time	Source	Destination	Protocol	Length	Info
267	73.497038200	127.0.0.1	127.0.0.1	TCP	56	9948 → 5058 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
268	73.497093800	127.0.0.1	127.0.0.1	TCP	56	5058 → 9948 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
269	73.497115500	127.0.0.1	127.0.0.1	TCP	44	9948 → 5058 [ACK] Seq=1 Ack=1 Win=65280 Len=0

รูปที่ 7.1 packet ของ TCP 3-way handshake ที่ได้จากการดักจับโดยใช้ wireshark



รูปที่ 7.2 Follow TCP stream ที่ได้จากการส่ง request ขอดูรายชื่อ webtoon ทั้งหมด